

Build an Eve PM Assistant

A Clean Start-to-Finish Mini-Course for Durable AI Agents, OpenAI, Custom Tools, and a Local Browser UI

[Workshop Asset](#) | [Instructor Guide](#) | [Copy/Paste Commands and Prompts](#)

What you will build

A local Eve agent that uses a direct OpenAI provider, custom Product Management tools, a terminal-based Eve TUI, and a simple browser UI for everyday use.

Module	Outcome
1. Setup	Create and run a working Eve agent locally.
2. Direct OpenAI	Use OPENAI_API_KEY without exposing secrets to the browser.
3. PM Tools	Add typed tools for feedback triage, scoring, PRDs, action plans, mock issue search, and decision logs.
4. Terminal Workflow	Test the agent and tools in the Eve TUI.
5. Browser UI	Add a simple local web chat UI that proxies to Eve.
6. Runbook	Run Eve and the UI with two terminal commands.

Table of contents

1. Course overview
 2. Architecture in plain English
 3. Prerequisites and Node 24 notes
 4. Build the terminal Eve agent
 5. Configure direct OpenAI safely
 6. Add the Product Management assistant tools
 7. Test the agent from the terminal
 8. Add a simple browser UI
 9. Run commands for daily use
 10. Teaching checklist and handoff notes
- Appendix A: Cursor prompt - PM assistant tools
- Appendix B: Cursor prompt - Browser UI

1. Course overview

Purpose. This mini-course teaches a clean, repeatable path for building a local Eve-based Product Management Assistant. It avoids the messy trial-and-error and focuses on the final successful workflow.

Audience. Builders, product managers, technical program managers, founders, and AI tinkerers who want to teach or reproduce a practical agentic workflow.

Instructor framing

This is not just a chatbot. The goal is to show how an agent can use typed tools, durable sessions, a terminal TUI, and a lightweight browser UI as a small operating system for product work.

Learning outcomes

- Scaffold and run an Eve agent locally.
- Use OpenAI directly with OPENAI_API_KEY instead of AI Gateway.
- Add custom Eve tools with typed inputs and predictable outputs.
- Test tool calls from the Eve terminal interface.
- Add a simple browser UI that proxies to the local Eve HTTP API.
- Run the completed project with two terminal commands.
- Teach the workflow clearly without exposing secrets or debugging history.

What the finished project contains

Area	Files or behavior
Agent	agent/agent.ts, agent/instructions.md
Tools	agent/tools/ingest_feedback.ts, score_opportunities.ts, generate_prd.ts, create_action_plan.ts, mock_linear_search.ts, save_decision_log.ts, risky_delete_or_archive.ts
Browser UI	web/server.ts, web/src/app.ts, web/public/index.html, web/public/styles.css, web/public/app.js
Docs	EVE_PM_ASSISTANT_README.md
Secrets	.env.local with OPENAI_API_KEY, ignored by Git

2. Architecture in plain English

Eve agent. The Eve runtime owns the agent loop, tool execution, durable session, and HTTP API. The same agent can be used from the terminal or through a browser UI.

Direct OpenAI provider. The agent calls OpenAI directly using @ai-sdk/openai and an OPENAI_API_KEY stored in .env.local. The key stays server-side.

Custom PM tools. Each tool lives under agent/tools/. The filename becomes the tool name the model can call. Tools use typed schemas and deterministic logic where possible.

Browser UI. The browser UI is separate from the agent. It calls a local UI server on port 3001, which proxies to the Eve HTTP API running on port 2000.

Mental model

```
Browser UI http://127.0.0.1:3001
  |
  | /eve/* proxy
  v
```

```
Eve HTTP API http://127.0.0.1:2000
    |
    | model call + tool calls
    v
OpenAI provider + agent/tools/*
```

Security rule

Never put OPENAI_API_KEY in browser code. Keep it in .env.local and make sure .env.local is ignored by Git.

3. Prerequisites and Node 24 notes

Assumption for this course

This tutorial assumes Node 24 is installed. On this Mac setup, Node 24 was installed through Homebrew as node@24 and run with a temporary PATH override. That approach avoids changing the global Node version used by other projects.

Confirm Node 24 is available

```
/opt/homebrew/opt/node@24/bin/node -v
# expected: v24.x.x
```

Why use a PATH override?

It lets you run this Eve project with Node 24 while leaving your normal node command untouched for other projects.

If someone has Node 23 active

If node -v shows Node 23, do not force-link Node 24 globally unless you want every project to use it. Use one of the safe options below.

Homebrew option

```
# Homebrew option: install Node 24 without replacing your global node
brew install node@24

# Use Node 24 for a single command
PATH="/opt/homebrew/opt/node@24/bin:$PATH" node -v
```

nvm option

```
# nvm option, if your learners use nvm
nvm install 24
nvm use 24
node -v
```

Recommended teaching shortcut. Throughout the course, run Eve commands using the PATH override so learners do not accidentally disturb other local projects.

4. Build the terminal Eve agent

Create a clean project folder

Scaffold the Eve app

```
cd ~/Desktop
```

```
mkdir -p eve
cd eve

PATH="/opt/homebrew/opt/node@24/bin:$PATH" npm_config_legacy_peer_deps=true npx --yes -p eve@latest -p
ai -p zod eve init my-agent

cd my-agent
```

The scaffold creates the Eve project, package.json, agent files, and the terminal interface. If the init command starts a dev process, stop it with Ctrl+C before editing files.

Clean-room note

Build the course project in its own folder. Do not add Eve into a production app until learners understand the runtime, model provider, and port behavior.

Install the OpenAI provider

Install direct OpenAI provider

```
PATH="/opt/homebrew/opt/node@24/bin:$PATH" npm install @ai-sdk/openai
```

5. Configure direct OpenAI safely

Set agent/agent.ts

Direct OpenAI model config

```
cat > agent/agent.ts <<'EOF'
import { defineAgent } from "eve";
import { openai } from "@ai-sdk/openai";

export default defineAgent({
  // Direct OpenAI provider. Requires OPENAI_API_KEY in .env.local.
  model: openai("gpt-5"),
});
EOF
```

The important distinction is that `openai("gpt-5")` is a provider-authored model call. A string such as `"openai/gpt-5"` or `"openai/gpt-5.5"` may route through a gateway-style configuration depending on the framework.

Create .env.local

Create and edit .env.local

```
cat > .env.local <<'EOF'
OPENAI_API_KEY=PASTE_YOUR_REAL_OPENAI_API_KEY_HERE
EOF

nano .env.local
```

Replace the placeholder with the real key. Do not include quotes and do not paste the key into course material, Git, browser code, or chat transcripts.

Protect secrets

```
grep -q '^\.env\.local$' .gitignore || echo ".env.local" >> .gitignore
```

```
git status --short --ignored | grep env
# expected: !! .env.local
```

6. Add the Product Management assistant tools

Use Cursor Agent mode to create the PM assistant behavior and custom tools. The full prompt is included in Appendix A. The tool set should be additive and should not expose secrets.

Tool	Purpose
ingest_feedback	Normalize raw feedback into categories, severity, and signals.
score_opportunities	Score opportunities with a RICE-style formula.
generate_prd	Create a structured PRD in Markdown.
create_action_plan	Turn a goal, constraints, and team into milestones.
mock_linear_search	Return mock issue data to simulate future Linear/GitHub/Jira integration.
save_decision_log	Save a decision record to the Eve sandbox workspace.
risky_delete_or_archive	Demonstrate human approval gating with a simulated destructive action.

Behavior rules that matter

The agent should call `ingest_feedback` for raw feedback, `score_opportunities` for prioritization, `generate_prd` for PRDs, `create_action_plan` for execution planning, `mock_linear_search` for issue searches, and `save_decision_log` for decisions.

Validate after tool creation

Typecheck

```
PATH="/opt/homebrew/opt/node@24/bin:$PATH" npm run typecheck
```

7. Test the agent from the terminal

Start the Eve terminal interface

Terminal workflow

```
cd /Users/sunnyside/Desktop/eve/my-agent
PATH="/opt/homebrew/opt/node@24/bin:$PATH" npm run dev
```

At the Eve prompt, send a simple message first. Then test the tool workflow directly.

Basic test prompt

```
hello
```

Tool test prompt

```
Use the ingest_feedback tool to normalize this feedback:
Source: customer interviews
Items:
1. Enterprise admins say onboarding is confusing.
2. Two users reported slow dashboard load times.
3. One prospect needs HubSpot integration before buying.
```

A successful tool test should show a line similar to:

Expected signal

```
✓ ingest_feedback source="customer interviews" items=[3]
→ normalizedItems=[3] countsByCategory={...} countsBySeverity={...}
```

Tool chaining test

```
Yes. Prioritize these and suggest next steps using a product-management framework.
```

8. Add a simple browser UI

The browser UI should be an additive layer. It should not refactor the model config or the PM tools. It should call the Eve HTTP API through a local proxy so secrets stay server-side.

Port model

In this working setup, Eve listens on `http://127.0.0.1:2000` and the browser UI listens on `http://127.0.0.1:3001`. The UI server proxies `/eve/*` to the Eve base URL.

Use the UI build prompt in Appendix B. The final UI should include a conversation area, textarea, Send button, Clear session button, debug toggle, and quick actions for common PM workflows.

Quick action	What it inserts
Triage feedback	A prompt that calls <code>ingest_feedback</code> with sample customer interviews.
Score opportunities	A RICE-style scoring prompt with four sample opportunities.
Draft PRD	A PRD prompt for an enterprise onboarding checklist.
Create action plan	A 3-week beta launch plan prompt.
Search mock issues	A mock issue search prompt.
Save decision log	A decision record prompt.

9. Run commands for daily use

Terminal 1 - Eve agent

Keep this running

```
cd /Users/sunnyside/Desktop/eve/my-agent
PATH="/opt/homebrew/opt/node@24/bin:$PATH" npm run dev
```

Terminal 2 - Browser UI

Keep this running

```
cd /Users/sunnyside/Desktop/eve/my-agent
EVE_BASE_URL=http://127.0.0.1:2000 PATH="/opt/homebrew/opt/node@24/bin:$PATH" npm run dev:ui
```

Open the browser UI at:

Local URL

```
http://127.0.0.1:3001
```

Validation commands

Validate

```
PATH="/opt/homebrew/opt/node@24/bin:$PATH" npm run typecheck
PATH="/opt/homebrew/opt/node@24/bin:$PATH" npm run build
PATH="/opt/homebrew/opt/node@24/bin:$PATH" npm run build:ui
```

Optional: confirm Eve HTTP API directly

Direct HTTP test

```
curl -i -X POST http://127.0.0.1:2000/eve/v1/session -H 'content-type: application/json' -d '{"message":"hello direct eve"}'
```

10. Teaching checklist and handoff notes

- Start by explaining the architecture: Eve runtime, tools, OpenAI provider, browser proxy.
- Show the terminal TUI first so learners see the agent is real before the browser UI exists.
- Explain why .env.local must stay ignored and why API keys never belong in frontend code.
- Demonstrate one direct tool call, then a chained workflow.
- Show the two-terminal run model and why EVE_BASE_URL points the UI to Eve.
- Use the debug panel only as a teaching aid; the normal user flow should not require it.

Demo script

1) Send hello in the browser UI. 2) Click Triage feedback and Send. 3) Ask it to prioritize the ingested feedback. 4) Click Draft PRD. 5) Save a decision log. This shows the full PM workflow.

Clean Git checklist

Pre-commit sanity check

```
git status --short

git status --short --ignored | grep env
# expected: !! .env.local

git diff -- agent/agent.ts agent/tools agent/instructions.md
```

Only commit the project files. Do not commit .env.local, API keys, temporary screenshots, or local debug artifacts.

Appendix A: Cursor prompt - PM assistant tools

Prompt A1 - Build the PM assistant tools

Copy/paste prompt

Paste this into Cursor Agent mode. Keep it as one prompt so Cursor has the full context.

You are in Agent mode. Build a working Eve-based Product Management Assistant prototype using the existing Eve scaffold.

Important constraints:

- Do not expose secrets.
- Do not commit changes unless asked.
- Preserve direct OpenAI setup if it already exists.
- Use TypeScript and Eve tool conventions.
- Keep the implementation simple, local, and easy to teach.

Goal:

Create an Eve agent that helps a product manager triage feedback, score opportunities, create PRDs, build action plans, search mock issue data, and save decision logs.

Tasks:

1. Update agent/instructions.md with a senior PM assistant system prompt.

The assistant should:

- Help triage customer feedback, feature requests, bugs, and stakeholder asks.
- Use tools when relevant.
- Avoid pretending to access real systems unless a tool provides the data.
- Clearly distinguish mock data from real data.
- Avoid invented exact calendar dates unless a date or planning window is provided.
- Use relative timing such as next 48 hours, this sprint, next sprint, or within 2 weeks when dates are missing.

2. Add tool routing rules:

- Raw feedback -> ingest_feedback.
- Prioritization -> score_opportunities.
- PRD request -> generate_prd.
- Execution plan -> create_action_plan.
- Issue or ticket search -> mock_linear_search unless a real integration exists.
- Save decision -> save_decision_log.

3. Create these tools under agent/tools/:

- ingest_feedback.ts
- score_opportunities.ts
- generate_prd.ts
- create_action_plan.ts
- mock_linear_search.ts
- save_decision_log.ts
- risky_delete_or_archive.ts

4. Tool details:

ingest_feedback:

- Accept source and items.
- Normalize feedback into category, severity, summary, and signals.
- Use deterministic keyword matching.
- Use snake_case signals such as activation_risk, usability_friction, performance_degradation, deal_blocker_potential, buyer_requirement, enterprise_segment, admin_workflow, revenue_blocker, reliability_risk, general_feedback.

score_opportunities:

- Accept title, description, reach, impact, confidence, effort, strategicFit, and risk.
- Score using: $(reach * impact * confidence) / \max(effort, 1) + strategicFit - risk$.
- Sort descending and return rationale.

generate_prd:

- Return Markdown PRD with Summary, Problem, Target Users, Goals, Non-Goals, Requirements, Success Metrics, Risks, Open Questions, and Next Steps.

create_action_plan:

- Return milestones, owners, tasks, risks, and immediate next steps.
- Use relative phases when no exact deadline is provided.

mock_linear_search:

- Return hardcoded mock PM issues.
- Filter by query, status, and limit.
- Clearly label results as mock data.

save_decision_log:

- Save a Markdown decision log under /workspace/decision-logs using ctx.getSandbox().
- Return saved, path, and markdown.

risky_delete_or_archive:

- Demonstrate human approval gating.
- Simulate the action only. Do not delete anything.

5. Add EVE_PM_ASSISTANT_README.md with:

- What was built.
- How to run locally.
- Required environment variables.
- Test prompts.
- Limitations.

6. Validate:

PATH="/opt/homebrew/opt/node@24/bin:\$PATH" npm run typecheck

Final response:

- List files changed.
- List tools added.
- Explain how to run and test.
- Confirm no secrets were exposed.

Appendix B: Cursor prompt - Browser UI

Prompt B1 - Add a simple browser UI

Copy/paste prompt

Paste this into Cursor Agent mode. Keep it as one prompt so Cursor has the full context.

You are in Agent mode. Build a simple browser UI for the existing Eve Product Management Assistant so it can be used from a page instead of the terminal TUI.

Critical constraints:

- The PM assistant tools and model configuration already work.
- Do not refactor tools.
- Do not modify agent/agent.ts.
- Do not modify .env.local.
- Do not modify agent/tools/*.
- Do not expose OPENAI_API_KEY in frontend code.
- Keep existing Eve terminal behavior working.
- Add only the browser UI layer and minimal proxy/server files needed.

Goal:

Create a simple local web chat UI where I can:

1. Type or paste a prompt.
2. Send it to the Eve agent.
3. See the assistant response.
4. Continue the same session using continuationToken.
5. Use quick-action buttons for common PM workflows.
6. Clear the session.
7. Open a debug panel when needed.

Architecture:

- Eve HTTP API runs at http://127.0.0.1:2000.
- Browser UI runs at http://127.0.0.1:3001.
- UI server proxies /eve/* to EVE_BASE_URL.
- EVE_BASE_URL defaults to http://127.0.0.1:2000 and can be overridden with an environment variable.

Required UI:

- Header: PM Assistant.
- Subtitle: Triage feedback, score opportunities, draft PRDs, and create action plans.
- Live connection status.
- Quick action buttons:
 1. Triage feedback
 2. Score opportunities
 3. Draft PRD
 4. Create action plan
 5. Search mock issues
 6. Save decision log
- Conversation area with distinct user and assistant messages.
- Textarea input.
- Send button.
- Clear session button.
- Show debug button.
- Loading state only while a request is active.

Quick action behavior:

- Buttons insert prompt templates into the textarea.
- Buttons do not auto-send.

Eve HTTP behavior:

- New session: POST /eve/v1/session with { message }.
- Store sessionId and continuationToken from the response.
- Stream with GET /eve/v1/session/:sessionId/stream.
- Follow-up: POST /eve/v1/session/:sessionId with continuationToken and message.
- Update continuationToken after each response.

Stream parsing requirements:

- Parse NDJSON robustly.
- Support event.type, event.event, event.name, event.kind, and event.data?.type.
- Extract text from Eve shapes:

- data.message as a plain string.
- data.messageSoFar.
- data.messageDelta.
- data.text, data.content, data.output, data.result, data.value.
- nested content arrays and parts arrays.
- For message.completed, display the final full string and clear loading.
- For message.appended, update the in-progress assistant message.
- For input.requested, display the prompt and option labels as an assistant message.
- For actions.requested or action.result, show a small tool line if a tool name is available.
- Clear loading on message.completed, result.completed, turn.completed, session.waiting, session.completed, turn.failed, session.failed, step.failed, fetch failure, timeout, and Clear session.
- Do not mutate the user message when assistant streaming starts. Create a separate assistant message and update only that assistant message.

Suggested files:

- web/server.ts
- web/src/app.ts
- web/public/index.html
- web/public/styles.css
- web/public/app.js generated by build:ui
- web/tsconfig.json

Package scripts:

- build:ui
- dev:ui
- Extend typecheck if needed.

Validation:

Run:

PATH="/opt/homebrew/opt/node@24/bin:\$PATH" npm run typecheck

PATH="/opt/homebrew/opt/node@24/bin:\$PATH" npm run build

PATH="/opt/homebrew/opt/node@24/bin:\$PATH" npm run build:ui

Manual run commands:

Terminal 1:

PATH="/opt/homebrew/opt/node@24/bin:\$PATH" npm run dev

Terminal 2:

EVE_BASE_URL=http://127.0.0.1:2000 PATH="/opt/homebrew/opt/node@24/bin:\$PATH" npm run dev:ui

Expected local URL:

http://127.0.0.1:3001

Final response:

- List files changed.
- Explain how to run the UI.
- Confirm no agent/model/tool files were changed.
- Confirm no secrets are exposed.

Appendix C: Quick test prompts

Basic response

```
hello
```

Feedback triage

```
Use the ingest_feedback tool to normalize this feedback:
Source: customer interviews
Items:
1. Enterprise admins say onboarding is confusing.
2. Two users reported slow dashboard load times.
3. One prospect needs HubSpot integration before buying.
```

Opportunity scoring

```
Use the score_opportunities tool to score these opportunities:
1. HubSpot integration - reach 40, impact 4, confidence 0.7, effort 6, strategicFit 3, risk 2
2. Faster dashboard loading - reach 300, impact 3, confidence 0.9, effort 3, strategicFit 4, risk 1
3. Self-serve onboarding checklist - reach 120, impact 4, confidence 0.8, effort 4, strategicFit 5, risk 1
4. Admin audit logs - reach 60, impact 3, confidence 0.6, effort 5, strategicFit 4, risk 2
```

PRD generation

```
Use the generate_prd tool to create a PRD for a self-serve onboarding checklist for new enterprise admins.
Problem: Admins are confused during initial setup and do not know which steps matter.
Audience: Enterprise admins.
Goals:
- Improve activation
- Reduce support tickets
- Help admins invite users and configure integrations
Requirements:
- Checklist
- Progress state
- Recommended next action
- Help links
- Admin completion analytics
Success metrics:
- Activation within 7 days
- Checklist completion
- Support ticket reduction
```

Action plan

```
Use the create_action_plan tool to create an execution plan.
Goal: Launch a beta for the self-serve onboarding checklist.
Deadline: 3 weeks from now.
Team:
- 1 product manager
- 1 engineer
- 1 designer
Constraints:
- Keep scope small
- Avoid backend-heavy work
- Must support enterprise admins first
Known risks:
- Onboarding requirements are not fully validated
- Engineering capacity is limited
```

Mock issue search

```
Use the mock_linear_search tool to search mock issues for onboarding and summarize what is blocking launch.
```

Decision log

Use the save_decision_log tool to save this product decision:

Title: Prioritize onboarding checklist over HubSpot integration

Decision: We are prioritizing the onboarding checklist first.

Context: Activation is the biggest current blocker, while HubSpot is important but only tied to one confirmed prospect so far.

Options considered:

- Build HubSpot integration first
- Improve dashboard performance first
- Build onboarding checklist first

Owner: Product

Final runbook

The three things to remember

```
# Terminal 1 - Eve agent
cd /Users/sunnyside/Desktop/eve/my-agent
PATH="/opt/homebrew/opt/node@24/bin:$PATH" npm run dev

# Terminal 2 - Browser UI
cd /Users/sunnyside/Desktop/eve/my-agent
EVE_BASE_URL=http://127.0.0.1:2000 PATH="/opt/homebrew/opt/node@24/bin:$PATH" npm run dev:ui

# Browser
http://127.0.0.1:3001
```

Course takeaway

You built a durable local agent, gave it PM-specific tools, validated it in the terminal, and wrapped it with a browser UI - without exposing secrets or burying the workflow in unnecessary complexity.